Phase 7 — LOG TIMING

Created	@March 2, 2026 2:50 PM
Tags	

ภาพรวมของ Log Timing Task

```
XCom (extract_sec, transform_sec, load_sec)
    ↓  ดึงค่าเวลาจาก XCom
    ↓  บันทึกลง CSV
/tmp/fraud_etl/timing_log_airflow.csv
```

STEP 1 : เพิ่ม Log Timing function

เพิ่ม function **ก่อน** DAG Definition:

```
# ============================================================
====================
# TASK 4 — LOG TIMING
# ============================================================
====================
def log_timing(dataset_key: str, **kwargs):
    ti = kwargs['ti']

        # รับค่าเวลาจาก XCom
    extract_sec   = ti.xcom_pull(key=f'extract_sec_{dataset
_key}')
    transform_sec = ti.xcom_pull(key=f'transform_sec_{datas
et_key}')
    load_sec      = ti.xcom_pull(key=f'load_sec_{dataset_ke
y}')

    # เขียน CSV
    write_header = not os.path.exists(CSV_LOG)
```

```
    with open(CSV_LOG, 'a', newline='') as f:
        writer = csv.writer(f)
        if write_header:
            writer.writerow(['dataset', 'extract_sec', 'tra
nsform_sec', 'load_sec'])
        writer.writerow([
            dataset_key,
            round(extract_sec,   4),
            round(transform_sec, 4),
            round(load_sec,      4)
        ])

    print(f'[Log] {dataset_key}: '
          f'extract={extract_sec:.3f}s | '
          f'transform={transform_sec:.3f}s | '
          f'load={load_sec:.3f}s')
```

STEP 2 : แก้ DAG Definition ให้ครบทุก Task

```
with DAG(
    dag_id='fraud_etl',
    default_args=default_args,
    schedule_interval=None,
    start_date=datetime(2024, 1, 1),
    catchup=False
) as dag:

    for key in ['10k', '100k', '1m']:
        extract_task = PythonOperator(
            task_id=f'extract_{key}',
            python_callable=extract,
            op_kwargs={'dataset_key': key}
        )
        transform_task = PythonOperator(
            task_id=f'transform_{key}',
            python_callable=transform,
            op_kwargs={'dataset_key': key}
```

```
        )
        load_task = PythonOperator(
            task_id=f'load_{key}',
            python_callable=load,
            op_kwargs={'dataset_key': key}
        )
        log_timing_task = PythonOperator(
            task_id=f'log_timing_{key}',
            python_callable=log_timing,
            op_kwargs={'dataset_key': key}
        )


        # Chain: extract → transform → load → log_timing
        extract_task >> transform_task >> load_task >> log_
timing_task
```

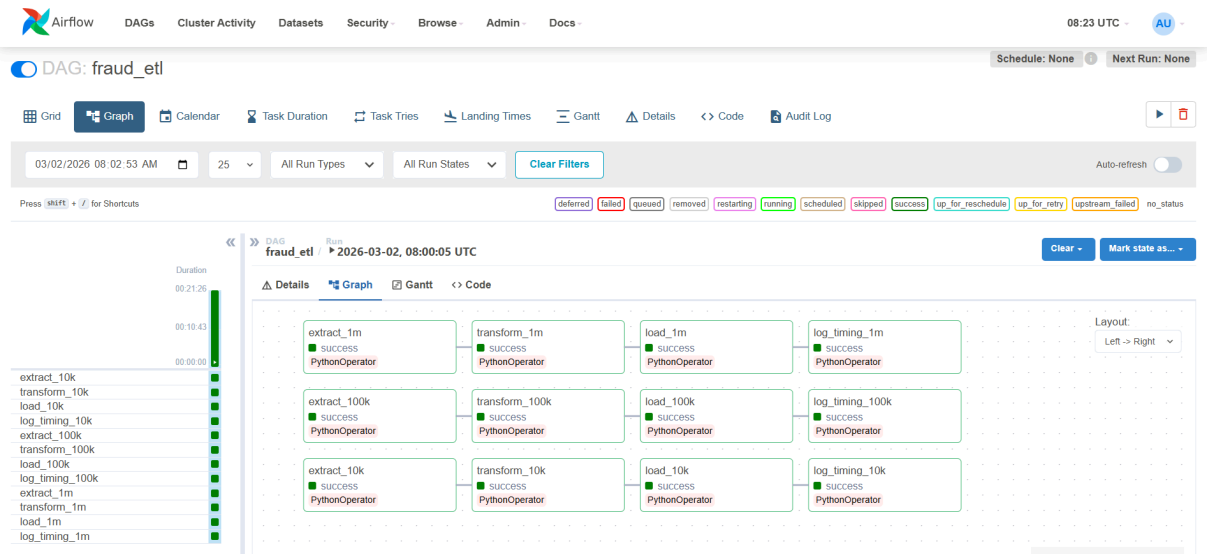
STEP 3 : ล้างข้อมูล MySQL ก่อน Trigger ใหม่

เพราะรันไปแล้ว 1,110,000 แถว ถ้า Trigger อีกครั้งจะซ้ำ:

```
USE fraud_dl;
TRUNCATE TABLE fraud_clean_airflow;
```

STEP 4 : Trigger DAG

กลับไปที่ `http://localhost:8080` แล้ว Trigger DAG



STEP 5 : ตรวจสอบข้อมูลใน MySQL

รัน SQL ใน MySQL :

```
USE fraud_dl;

-- นับแถวทั้งหมด
SELECT COUNT(*) AS total_rows FROM fraud_clean_airflow;

-- ดูตัวอย่างข้อมูล
SELECT * FROM fraud_clean_airflow LIMIT 5;

-- ตรวจว่า hash ถูกต้อง
SELECT
    cc_hash,
    first_hash,
    last_hash,
    dob_hash
FROM fraud_clean_airflow LIMIT 5;

-- ตรวจ amt ว่าไม่มี NULL หรือ <= 0
SELECT COUNT(*) AS bad_amt
FROM fraud_clean_airflow
WHERE amt IS NULL OR amt <= 0;
```

-

STEP 6 : ตรวจสอบผลลัพธ์

ดู CSV หลังรันเสร็จ:

```
cat /tmp/fraud_etl/timing_log_airflow.csv
```

ผลที่ได้:

```
dataset,extract_sec,transform_sec,load_sec
10k,0.746,20.3738,32.1863
100k,2.4046,24.1725,108.3891
1m,19.3615,38.9669,955.3723
```